

E-Commerce Platform for ShopEase Ltd.

Python

```
import re

from abc import ABC, abstractmethod

from dataclasses import dataclass, field

from datetime import datetime

from typing import Dict, List, Optional


# =====

# 1. VALUE OBJECTS & IMMUTABILITY

# =====

# Implements Data Immutability for Transactions and Value Objects (Martin 2017).


@dataclass(frozen=True)

class Product:

    """Immutable representation of a Kenyan beauty product."""

    product_id: str

    name: str

    category: str

    price_kes: float


@dataclass(frozen=True)

class TransactionRecord:
```

```

    """Immutable transaction log for security and auditing (Martin 2017)."""

    transaction_id: str

    order_id: str

    amount_kes: float

    payment_method: str

    timestamp: datetime = field(default_factory=datetime.now)


# =====

# 2. OBSERVER PATTERN (Notifications)

# =====

# Implements the Observer Pattern for decoupling notifications (Gamma et al.
1994).


class NotificationObserver(ABC):

    @abstractmethod

    def update(self, message: str) -> None:

        pass


class EmailNotificationService(NotificationObserver):

    def update(self, message: str) -> None:

        print(f"[NOTIFICATION][EMAIL] {message}")


class SmsNotificationService(NotificationObserver):

```

```

def update(self, message: str) -> None:

    print(f"[NOTIFICATION][SMS] {message}")


# =====

# 3. STRATEGY PATTERN (Payment Methods)

# =====

# Implements Strategy Pattern to ensure Extensibility for new payment methods
(Gamma et al. 1994).


class PaymentStrategy(ABC):

    @abstractmethod

    def pay(self, amount: float) -> bool:

        pass


class CashPayment(PaymentStrategy):

    def pay(self, amount: float) -> bool:

        print(f"Processing cash payment of KES {amount:,.2f} upon delivery.")

        return True


class MpesaPayment(PaymentStrategy):

    def __init__(self, phone_number: str):

        self.__phone_number = phone_number

```

```

def pay(self, amount: float) -> bool:

    print(f"Sending STK Push to {self.__phone_number} for KES {amount:,.2f}
via M-Pesa...")

    print("M-Pesa Transaction confirmed successfully.")

    return True

```

```

class CardPayment(PaymentStrategy):

```

```

    def __init__(self, card_number: str, card_type: str):

        self.__card_number = card_number

        self.__card_type = card_type # Debit or Credit

    def pay(self, amount: float) -> bool:

        masked_card = f"****-****-****-{self.__card_number[-4:]}"

        print(f"Processing {self.__card_type} card {masked_card} for KES
{amount:,.2f}...")

        return True

```

Extensible: Easily added Voucher payment without altering existing processing code

```

class VoucherPayment(PaymentStrategy):

```

```

    def __init__(self, voucher_code: str):

        self.__voucher_code = voucher_code

    def pay(self, amount: float) -> bool:

```

```

        print(f"Validating shopping voucher '{self.__voucher_code}' for KES
{amount:,.2f}...")

        return True

```

```

# =====

# 4. DATA ACCESS LAYER (Database Mock)

# =====

# Encapsulates data storage and retrieval (Martin 2017).

```

```

class DataAccessLayer:

    def __init__(self):

        # In-memory collections mimicking database tables

        self.__users: Dict[str, dict] = {}

        self.__products: Dict[str, Product] = {}

        self.__transactions: List[TransactionRecord] = []

    def save_user(self, user_id: str, user_data: dict) -> None:

        self.__users[user_id] = user_data

    def get_user(self, user_id: str) -> Optional[dict]:

        return self.__users.get(user_id)

    def save_product(self, product: Product) -> None:

```

```

        self.__products[product.product_id] = product

def get_all_products(self) -> List[Product]:

    return list(self.__products.values())

def log_transaction(self, record: TransactionRecord) -> None:

    self.__transactions.append(record)

    print(f"[DAL] Transaction {record.transaction_id} securely saved to
database.")

# =====

# 5. BUSINESS LOGIC LAYER

# =====

# Contains core e-commerce logic, leveraging Dependency Injection (Martin 2017).

class UserManagementService:

    """Handles authentication and registration with explicit security
constraints."""

    def __init__(self, dal: DataAccessLayer):

        self.__dal = dal # Dependency Injection

    def register_user(self, email: str, password: str, role: str) -> bool:

        # Validate User ID (Must be an email address)

```

```

if not re.match(r"^[^@]+@[^@]+\.[^@]+", email):

    print("Registration Failed: User ID must be a valid email address.")

    return False


# Validate Password (Min 8 mixed characters - requires letters and numbers
here)

if len(password) < 8 or password.isalpha() or password.isdigit():

    print("Registration Failed: Password must be at least 8 characters
long and contain mixed characters.")

    return False


if self.__dal.get_user(email):

    print("Registration Failed: User already exists.")

    return False


# Encapsulation: Store details in a dictionary structure mimicking an
encrypted DB row

self.__dal.save_user(email, {"password": password, "role": role})

print(f"User {email} ({role}) registered successfully.")

return True


def authenticate(self, email: str, password: str) -> bool:

    user = self.__dal.get_user(email)

    if user and user["password"] == password:

```

```

        print(f"User {email} authenticated successfully.")

        return True

    print("Authentication Failed: Invalid credentials.")

    return False


class OrderProcessingService:

    """Handles order checkouts and triggers payments/notifications using
    patterns."""

    def __init__(self, dal: DataAccessLayer):

        self.__dal = dal # Dependency Injection

        self.__observers: List[NotificationObserver] = []

    def attach_notification_service(self, observer: NotificationObserver) -> None:

        self.__observers.append(observer)

    def __notify_all(self, message: str) -> None:

        for observer in self.__observers:

            observer.update(message)

    def checkout(self, order_id: str, shopper_email: str, items: List[Product],
payment_strategy: PaymentStrategy) -> bool:

        total_amount = sum(item.price_kes for item in items)

        print(f"\n--- Processing Order {order_id} for {shopper_email} ---")

```



```

print(f"Total Amount: KES {total_amount:,.2f}")

# Execute Strategy Pattern for payment (Gamma et al. 1994)

payment_success = payment_strategy.pay(total_amount)

if payment_success:

    # Create Immutability Record

    tx_id = f"TX-{datetime.now().strftime('%Y%m%d%H%M%S')}"

    record = TransactionRecord(

        transaction_id=tx_id,

        order_id=order_id,

        amount_kes=total_amount,

        payment_method=payment_strategy.__class__.__name__

    )

    self.__dal.log_transaction(record)

    # Trigger Observer Pattern Notifications

    self.__notify_all(f"Order {order_id} successfully processed via

{payment_strategy.__class__.__name__}. Total: KES {total_amount:,.2f}")

    return True

print("Checkout failed due to payment complications.")

return False

```

```

# =====

# 6. PRESENTATION LAYER & RUNTIME SIMULATION

# =====

if __name__ == "__main__":

    print("=====")

    print("    SHOPEASE E-COMMERCE PLATFORM SYSTEM START    ")

    print("=====\\n")


    # Initialize Data Access Layer (Database Context)

    db = DataAccessLayer()


    # Seed Platform with Kenyan Beauty Products

    db.save_product(Product("P001", "Suzie Beauty Liquid Foundation", "Makeup",
2500.00))

    db.save_product(Product("P002", "Lavy Avocado Skin Glow Oil", "Skincare",
1800.00))

    db.save_product(Product("P003", "Shea Butter Hair Treatment (Kenya)",
"Haircare", 1200.00))


    # Initialize Services via Dependency Injection

    user_service = UserManagementService(db)

    order_service = OrderProcessingService(db)

```

```

# Attach Notification Observers (Observer Pattern)

order_service.attach_notification_service(EmailNotificationService())

order_service.attach_notification_service(SmsNotificationService())


# --- Scenario 1: User Registrations & Security Validations ---

print("--- 1. Testing Registration Guardrails ---")

# Invalid Email Violation

user_service.register_user("sharon_shopper", "Pass1234", "Shopper")

# Weak Password Violation (Strictly digits, under 8 chars)

user_service.register_user("sharon@gmail.com", "12345", "Shopper")

# Successful Valid Registration

user_service.register_user("sharon@gmail.com", "SecurePass56", "Shopper")

user_service.register_user("supplier_beauty@co.ke", "Supplier99!", "Supplier")

print()


# --- Scenario 2: User Login Authentication ---

print("--- 2. Testing Authentication ---")

is_logged_in = user_service.authenticate("sharon@gmail.com", "SecurePass56")

print()


# --- Scenario 3: Shopping and Order Checkout processing ---

if is_logged_in:

    print("--- 3. Simulation of Shopping Basket & Checkout Processes ---")

    # Fetching inventory items from Catalog

```

```

available_products = db.get_all_products()

# User adds items to their shopping cart

cart = [available_products[0], available_products[1]] # Foundation + Glow
Oil

# Scenario 3a: Shopper pays using Mpesa (Strategy 1)

mpesa_method = MpesaPayment(phone_number="+254712345678")

order_service.checkout("ORD-001", "sharon@gmail.com", cart, mpesa_method)

# Scenario 3b: Shopper performs another transaction using Card Payment
(Strategy 2)

cart_2 = [available_products[2]] # Hair Treatment

card_method = CardPayment(card_number="4000123456789010",
card_type="Debit")

order_service.checkout("ORD-002", "sharon@gmail.com", cart_2, card_method)

# Scenario 3c: Extensibility Test using a Shopping Voucher (Strategy 3)

cart_3 = [available_products[0]]

voucher_method = VoucherPayment(voucher_code="EASTER-GLOW-2026")

order_service.checkout("ORD-003", "sharon@gmail.com", cart_3,
voucher_method)

```

Observations on the Design

Working through this ShopEase Ltd. e-commerce platform, I really appreciated how the layered architecture came together in practice. The Presentation Layer feels natural in the `__main__` block—it simulates user interactions without cluttering the core logic. What stood out for me was the clean separation in the Business Logic Layer, where classes like `UserManagementService` and `OrderProcessingService` handle the real domain work while staying nicely encapsulated. I especially liked the Data Access Layer—by isolating it, the code avoids tight coupling to any specific database, which makes the whole system feel more flexible and maintainable.

On the design choices, the Strategy Pattern for payments immediately clicked for me. Seeing how easily I could add `VoucherPayment` without touching the checkout logic showed real extensibility. The Observer Pattern for notifications also felt elegant; attaching email and SMS observers (and imagining WhatsApp as a future one) highlights how decoupled the communication is. Using `@dataclass(frozen=True)` for `Product` and `TransactionRecord` gave me confidence in the immutability—transactions feel secure and tamper-proof. The

security validations, though basic, effectively enforce sensible rules for emails and strong passwords.

Reflections

Building this project helped me see how OOP patterns aren't just textbook concepts—they genuinely support cleaner, more adaptable code in a realistic setting like a Kenyan beauty products store with M-Pesa. Applying ideas from Clean Architecture made me value separation of concerns more deeply, especially for future scalability. The practical demo strengthened my understanding, but I now realize how much more robust it could become with real persistence, proper tests, and maybe some domain-driven design touches. Overall, this exercise reinforced my appreciation for thoughtful software craftsmanship and left me excited to apply these patterns in my own work.

References

Gamma, E., Helm, R., Johnson, R. and Vlissides, J., 1994. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley

Martin, R.C., 2017. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.